

Sunbeam Institute of Information Technology
Pune and Karad

Algorithms and Data structures

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

Merge sort

1. Divide array in two parts
2. Sort both partitions individually (by merge sort only)
3. Merge sorted partitions into temporary array
4. Overwrite temporary array into original array

$$mid = \frac{left + right}{2}$$

LP : left → mid

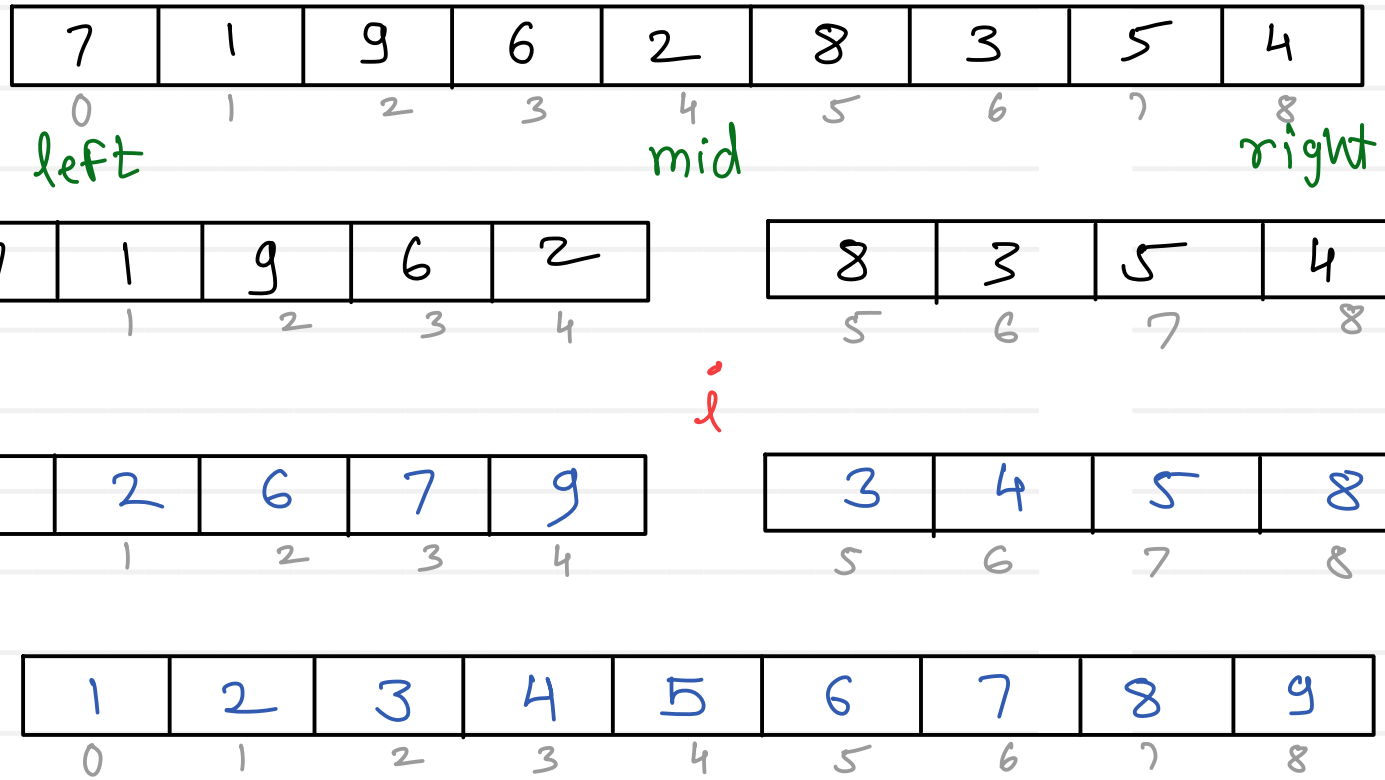
RP : mid+1 → right

size of temp array = right - left + 1

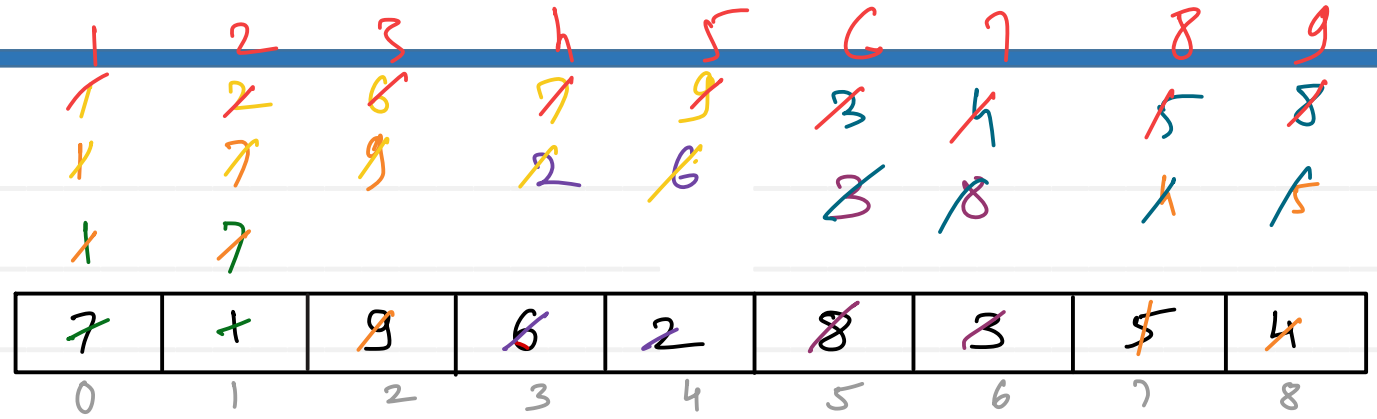
i = left → mid

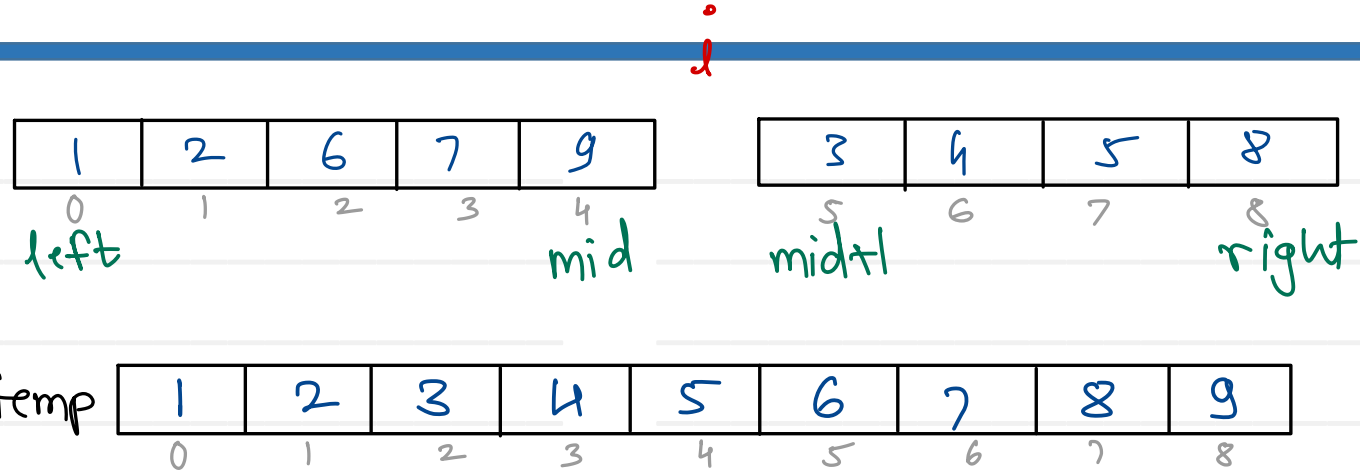
j = mid+1 → right

k = 0 → size-1



Merge sort


1 2 3 4 5 6 7 8 9
7 + 9 6 2 8 3 5 4
0 1 2 3 4 5 6 7 8
ms(arr, 0, 8) m = 4
1 2 3 4 5 6 7 8 9
0 1 2 3 4 5 6 7 8
ms(arr, 0, 4) m = 2
1 2 6 7 9
0 1 2 3 4
ms(arr, 5, 8) m = 6
3 4 5 8
0 1 2 3
ms(arr, 0, 2) m = 1
1 7 9
0 1 2
ms(arr, 3, 4) m = 3
2 6
0 1
ms(arr, 5, 6) m = 5
3 8
0 1
ms(arr, 7, 8) m = 7
4 5
0 1
ms(arr, 0, 1) m = 0
1 7
0 1
ms(arr, 2, 2)
ms(arr, 3, 3)
ms(arr, 4, 4)
ms(arr, 5, 5)
ms(arr, 6, 6)
ms(arr, 7, 7)
ms(arr, 8, 8)
ms(arr, 0, 0)
ms(arr, 1, 1)




```
while( i <= mid && j <= right) {
    if(arr[i] < arr[j]) {
        temp[k] = arr[i];
        k++; i++;
    }else {
        temp[k] = arr[j];
        k++; j++;
    }
}
```

```
while( i <= mid) {
    temp[k] = arr[i];
    k++; i++;
}
while( j <= right) {
    temp[k] = arr[j];
    k++; j++;
}
```

per level n elements will be compared

comps per level $\approx n$

No. of levels of division $\approx \log n$

total No. of comps = $n \log n$

Time $\propto$ Comps

Time $\propto n \log n$

Best
Avg
worst

$$T(n) = O(n \log n)$$

per level temp array of size n will be needed for merging

Auxilliary space $\approx n$

$$S(n) = O(n)$$

in place sorting

- sorting is done in place of array

e.g. selection, bubble, insertion, quick, heap sort

out place sorting

- sorting is done out side the place of array (eg. temp array)

e.g. merge sort

Quick sort

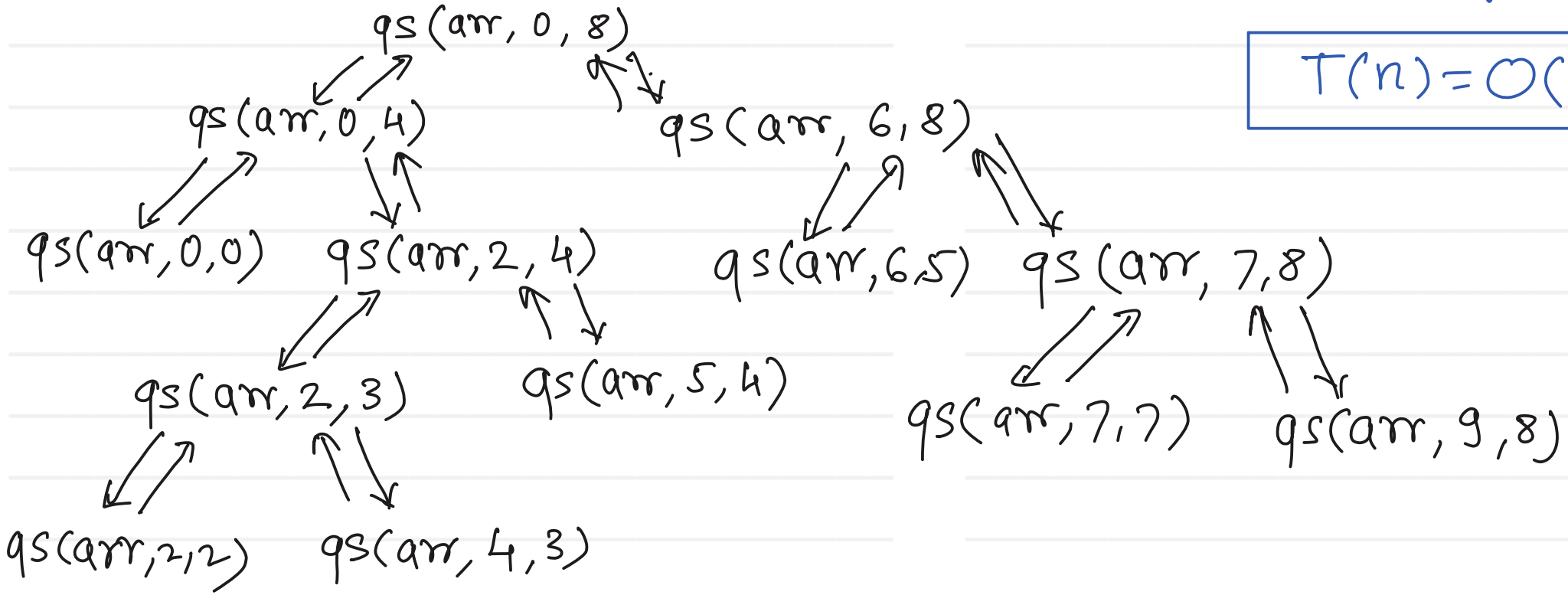
1. Select pivot/axis/reference element from array
2. Arrange lesser elements on left side of pivot
3. Arrange greater elements on right side of pivot
4. Sort left and right side of pivot again (by quick sort)

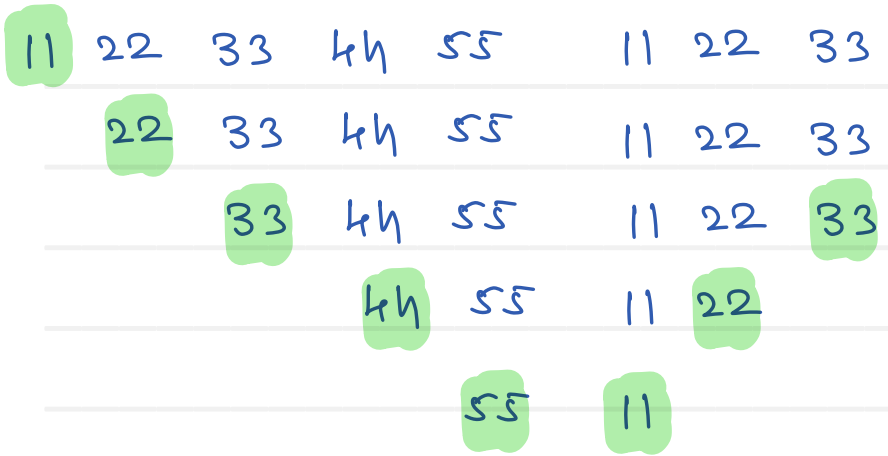
No. of levels of division = log n

comps per levels ≈ n

total comps = n log n

$$T(n) = O(n \log n)$$ Avg Best





No. of levels $\approx n$

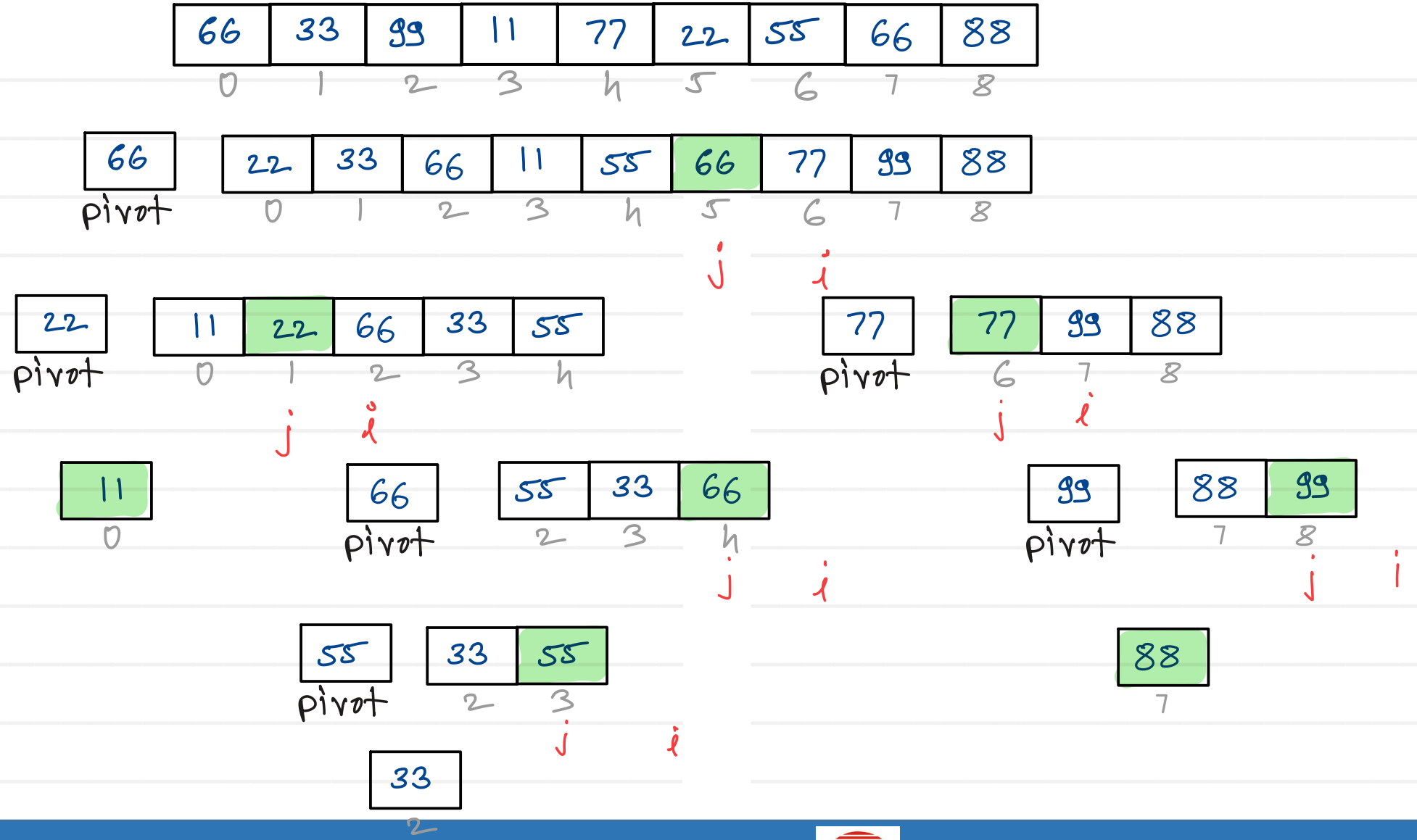
comps = $n * n$

$T(n) = O(n^2)$ worst



- time complexity is dependent on selection of pivot
- pivot is selected by any one of the below method
  1. extreme left
  2. extreme right
  3. mid
  3. median of three
  4. median of five
  5. dual pivot

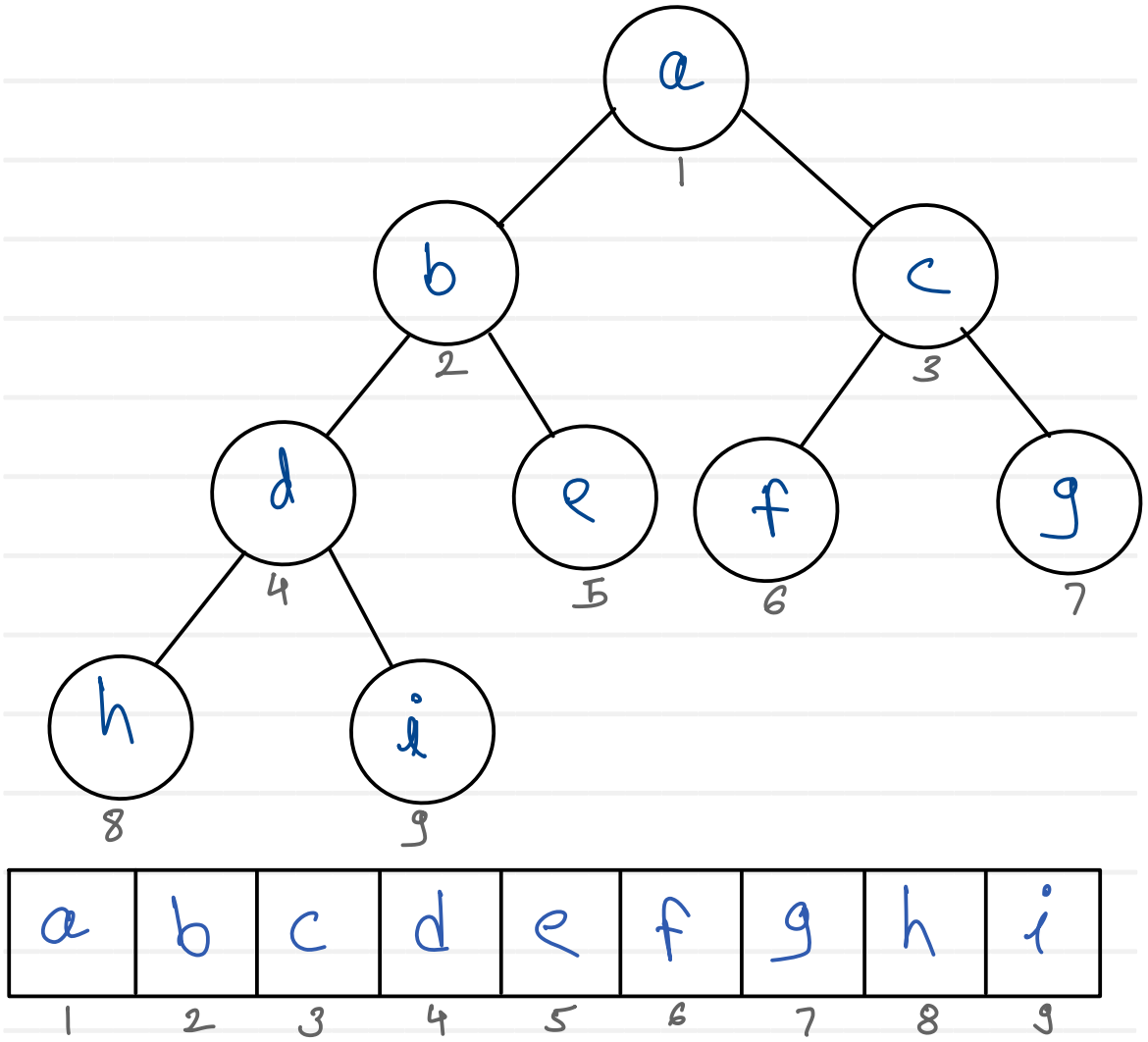
Quick sort


66 33 99 11 77 22 55 66 88
0 1 2 3 4 5 6 7 8
66
pivot
22 33 66 11 55 66 77 99 88
0 1 2 3 4 5 6 7 8
j
i
22
pivot
11 22 66 33 55
0 1 2 3 4
j
i
77
pivot
77 99 88
6 7 8
j
i
11
0
66
pivot
55 33 66
2 3 4
j
i
99
pivot
88 99
7 8
j
i
55
pivot
33 55
2 3
j
i
33
2
88
7

	space	Time Best	Time Avg	Time Worst
selection sort	O(1)	$O(n^2)$	$O(n^2)$	$O(n^2)$
bubble sort	O(1)	$O(n)$	$O(n^2)$	$O(n^2)$
insertion sort	O(1)	$O(n)$	$O(n^2)$	$O(n^2)$
Heap sort	O(1)	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick sort	O(1)	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Merge sort	O(n)	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

inplace sorting algorithm

Complete Binary Tree or Heap



- Complete Binary Tree (height = h)
- All levels should be completely filled except last
- All leaf nodes must be at level h or h-1
- All leaf nodes at level h must aligned as left as possible (filled from left to right without keeping any gap)
- Array implementation of Complete Binary Tree is called as heap

- array indices are used to maintain parent child relationship

node - i^{th} index

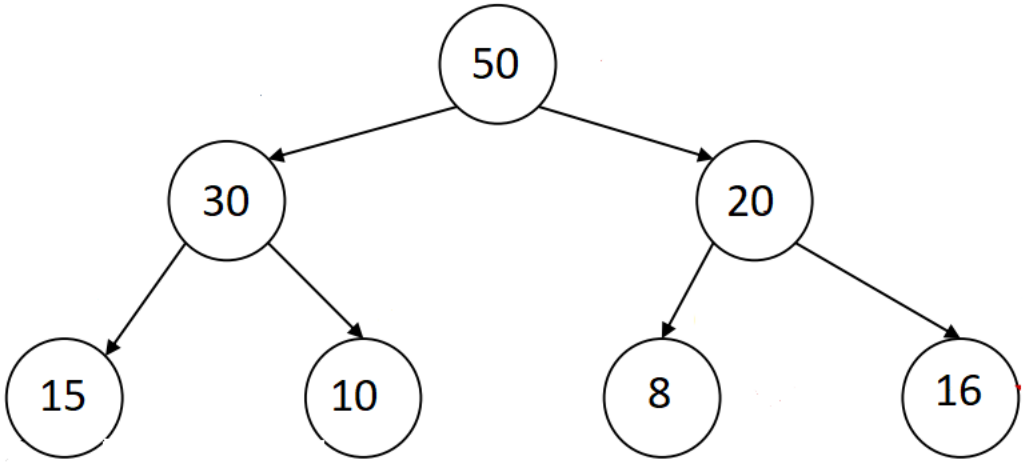
parent - $i/2$ index

left child - $i*2$ index

right child - $i*2+1$ index

Heap Types – Max and Min

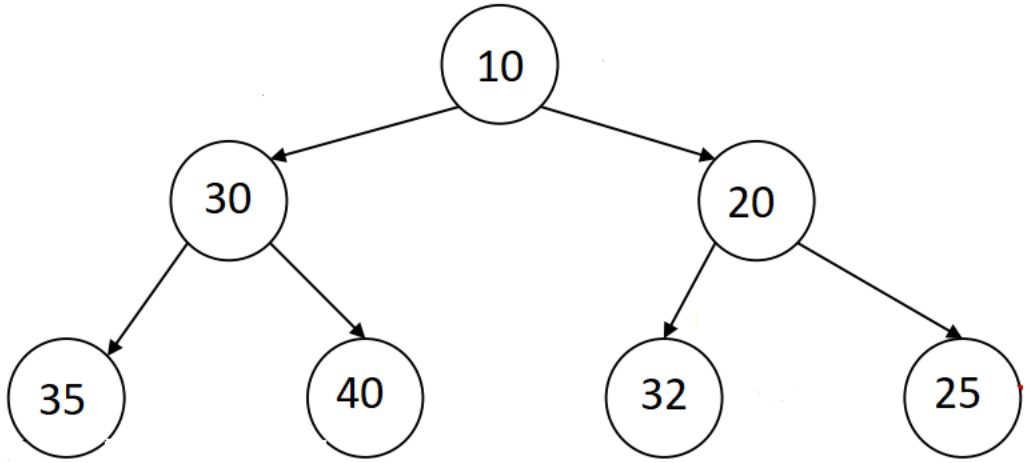
Max Heap



50	30	20	15	10	8	16
1	2	3	4	5	6	7

- Max heap is a heap data structure in which each node is greater than both of its child nodes.

Min Heap

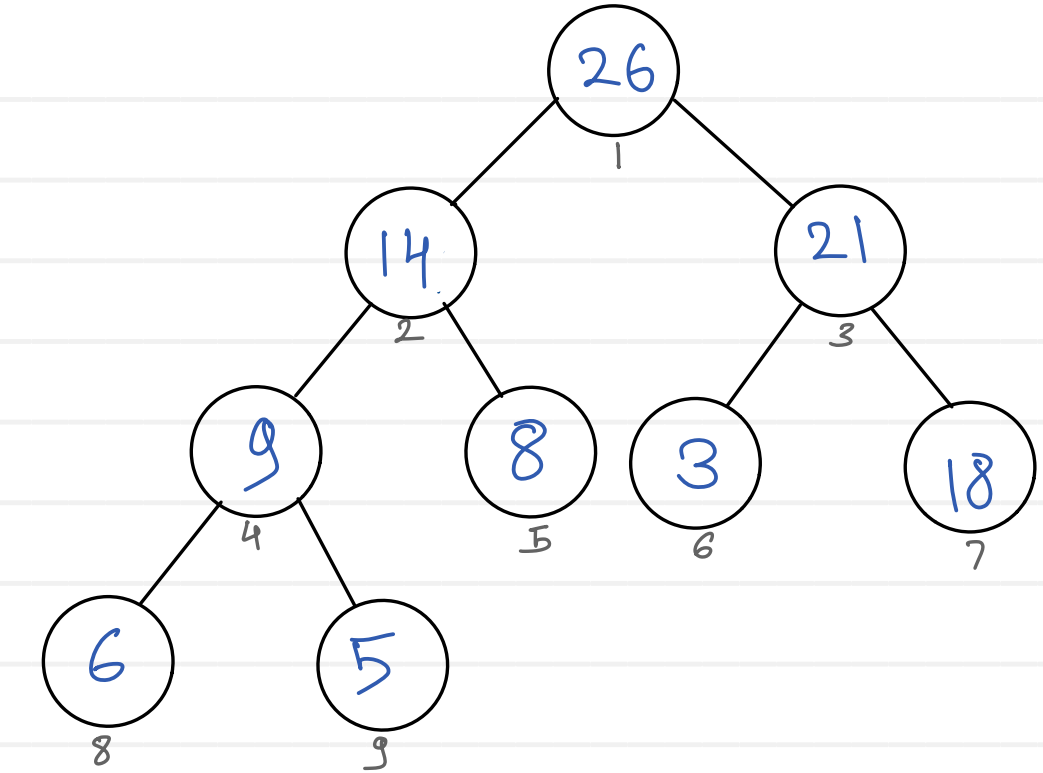


10	30	20	35	40	32	25
1	2	3	4	5	6	7

- Min heap is a heap data structure in which each node is smaller than both of its child nodes.

Heap - Create heap (Add)

(Max)



26	14	21	9	8	3	18	6	5
1	2	3	4	5	6	7	8	9

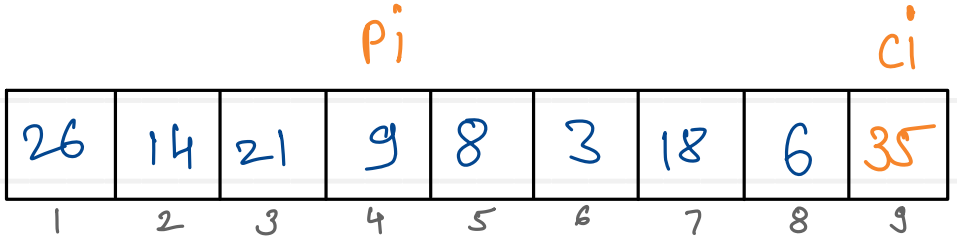
Keys : 6, 14, 3, 26, 8, 18, 21, 9, 5

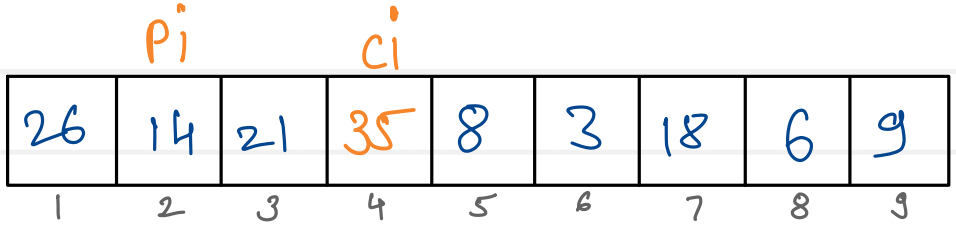
Algorithm :

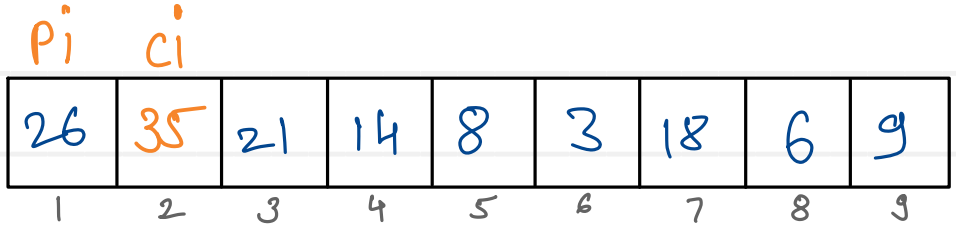
1. add new value at first empty index from left side
2. adjust position of newly added value by comparing it with all its ancestors one by one.

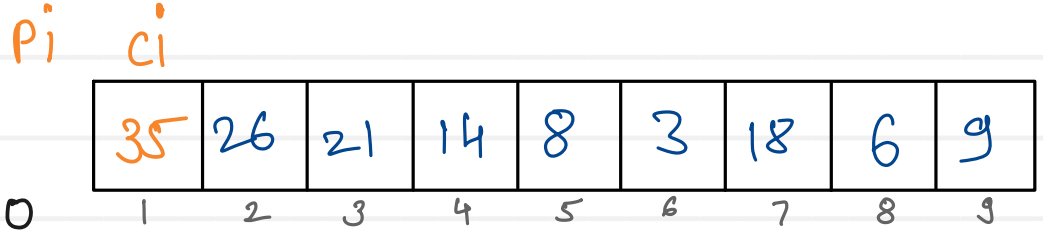
- to adjust position of newly added value, need to traverse from leaf to root position.

$$T(n) = O(\log n)$$









value = 35

Ci	Pi
9	4
4	2
2	1
1	0 ← invalid

ci = 9.

pi = ci / 2;

while(parent index is valid) {

if parent is maximum than child,

break;

if parent is minimum than child,

swap parent & child.

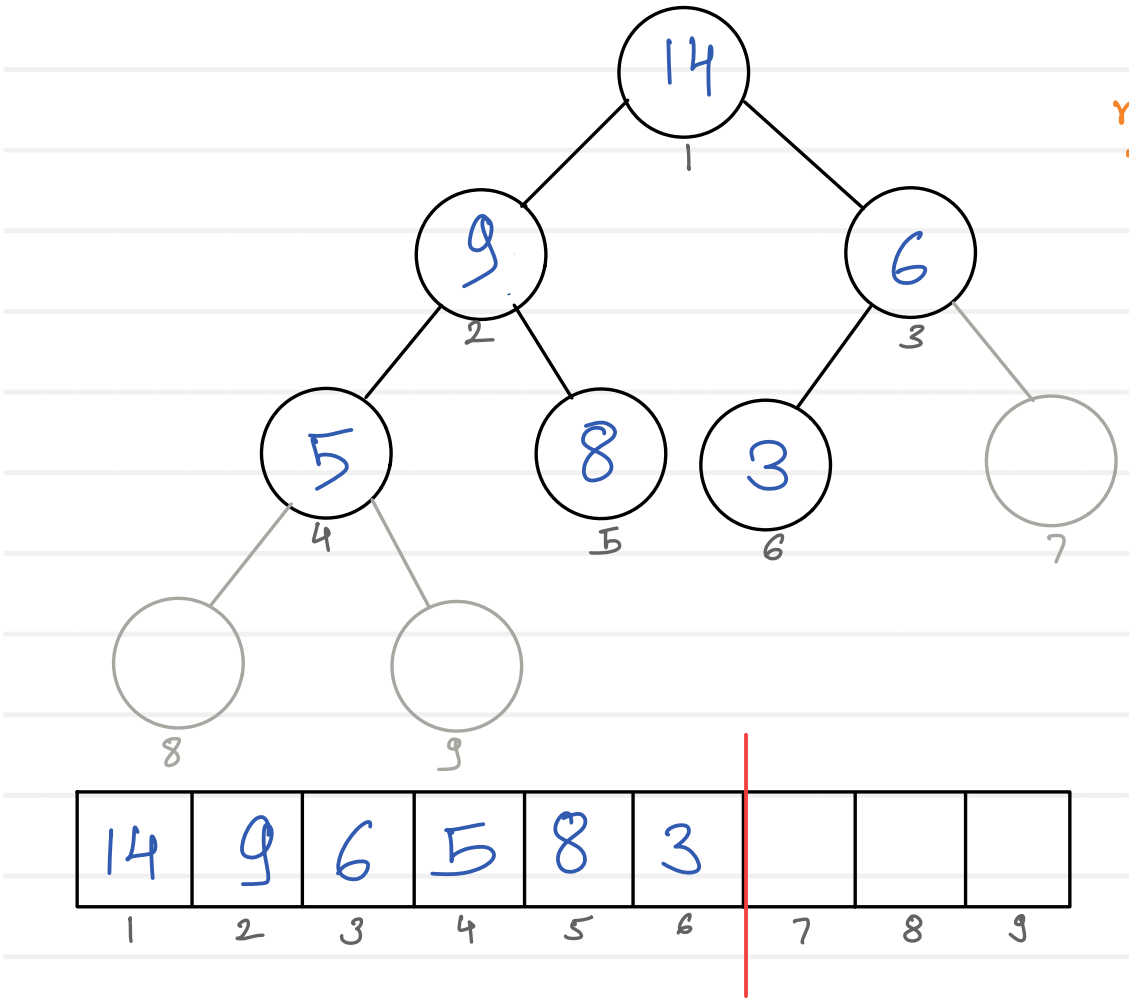
updata parent child index

ci = pi

pi = ci/2

}

Heap - Delete heap (Delete)



max value

26

21

18

Property : can delete only root node from heap

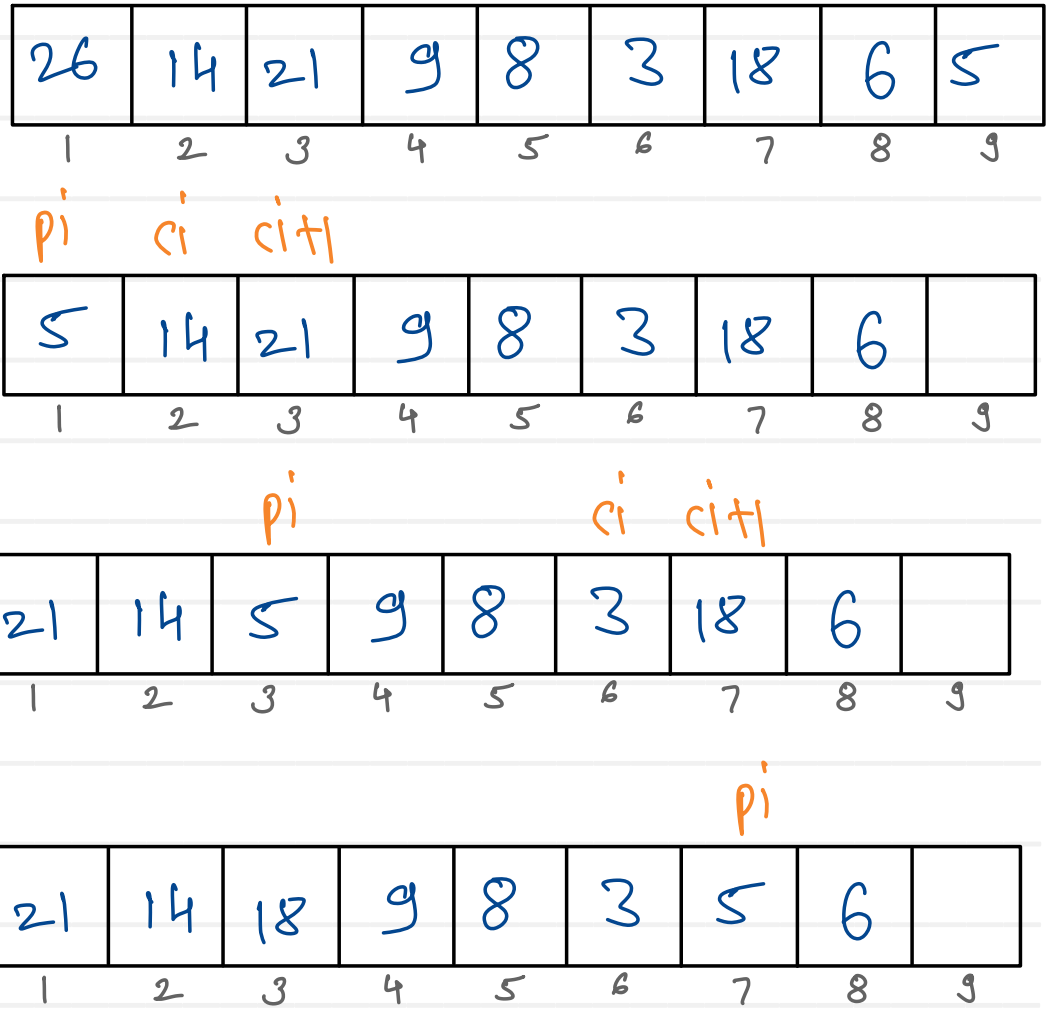
1. in max heap, always maximum element will be deleted from heap.
2. in min heap, always minimum element will be deleted from heap.

Algorithm :

1. promote last element of heap to the root place
2. adjust position of promoted value by comparing it with all its descendents one by one.

- to adjust position of promoted value, need to traverse from root to leaf position

$$T(n) = O(\log n)$$



max = 26 , size = 9

left child, right child

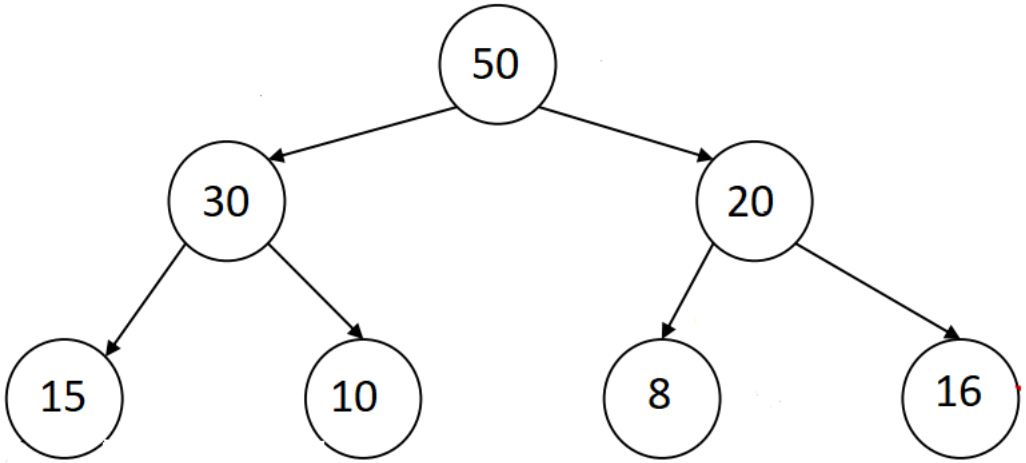
pi	ci	ci+1
1	2	3
3	6	7
7	14	← invalid child index

size = 8

```
pi = 1 ;
ci = pi * 2;
while ( child index is valid ) {
    find index of maximum child
    if parent is greater than max child
        break
    if parent is smaller than max child
        swap parent & max child
    updata parent & child index
        pi = ci ;
        ci = pi * 2;
}
```

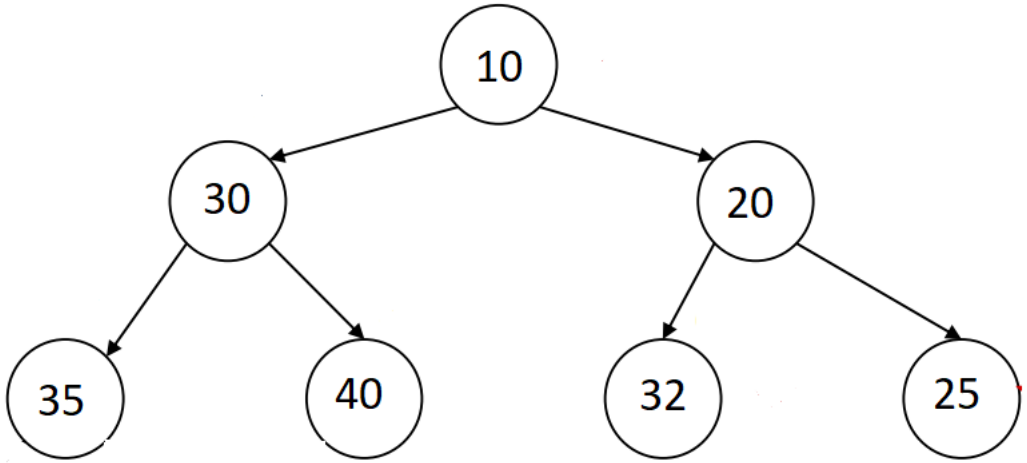
Priority Queues

Higher number
Higher priority



- In Max heap always root element which has highest value is removed

Lower number
Higher priority



- In Min heap always root element which has lowest value is removed

Priority Queue

- Always high priority element is deleted from queue
- value (priority) is assigned to each element of queue
- priority queue can be implemented using array or linked list.
- to search high priority data (element) need to traverse array or linked list
- Time complexity = O(n)

- priority queue can also be implemented using heap. because, maximum / minimum value is kept at root position in max heap & min heap respectively.
- push, pop & peek will be performed efficiently

max value → high priority → max heap
min value → high priority → min heap

Priority Queue

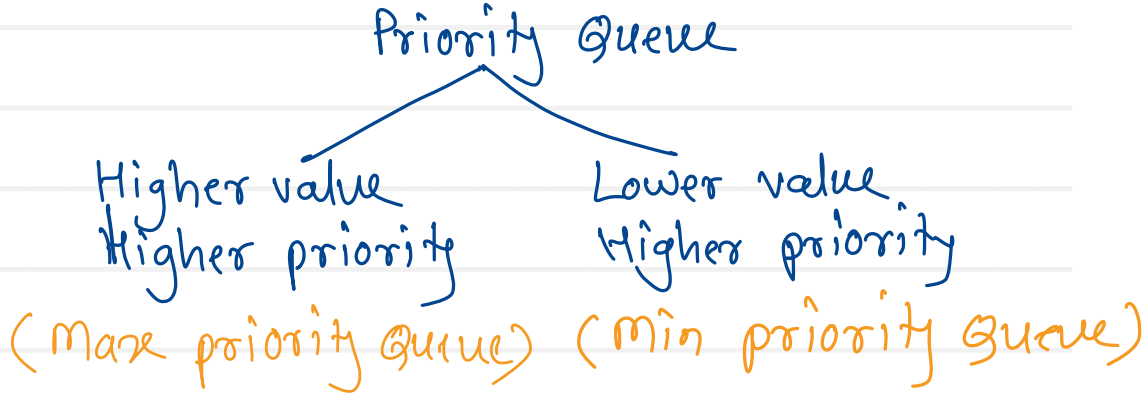
- Queue where high priority data is always peeked or deleted.
- Priority can be implemented using array, linked list and heap data structures.
- An array is used to store a value and its associated priority. In some simpler implementations, the value itself might represent the priority (e.g., lower value means higher priority).
- Array and linked list implementation of priority queue often leads to less efficient performance compared to heap-based implementations for insertion and deletion operations.

- **Ordered vs. Unordered Array**
  - **Ordered Array** / linked list
    - Elements are kept sorted by priority. Insertion requires shifting existing elements to maintain order, leading to O(n) time complexity for insertion.
    - Deletion of the highest priority element is O(1) as it's typically at the beginning or end of the array.
  - **Unordered Array:** / linked list
    - Elements are inserted without regard to order, making insertion O(1).
    - Deletion of the highest priority element requires searching the entire array to find it, resulting in O(n) time complexity for deletion.

Priority Queue Implementation

- Priority : number associated with value
- Priority range is defined by programmer

  e.g. Priority range : 1 to 10



- Every element of priority queue will have two parts:

  value : any data type

  priority : integer

```
struct item {
    int value;
    int priority;
};
```

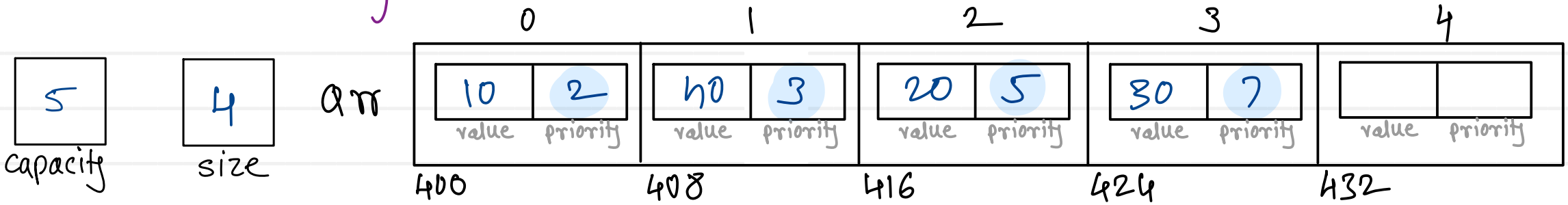
```
struct priorityQueue {
    struct item arr[5];
    int capacity;      (maxSize)
    int size;
};
```

Operations :

1) Enqueue
2) Dequeue
3) Peek
4) isEmpty → size == 0
5) isFull → size == capacity

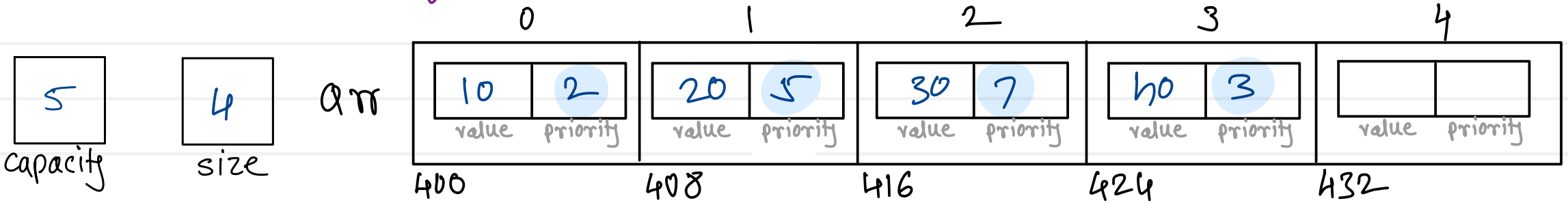
Priority Queue Implementation

Ordered array:



insert - O(n) delete - O(1) peek - O(1)
for shifting - O(n)

Unordered array:



insert - O(1) delete - O(n) peek - O(n)
for shifting - O(n)

Heap sort

arr	6	14	3	26	8	18	21	9	5
	1	2	3	4	5	6	7	8	9

Algorithm :

1. convert given array into heap
   max heap (ascending) or min heap (descending)
2. delete elements from heap one by one and keep them into array from right side.

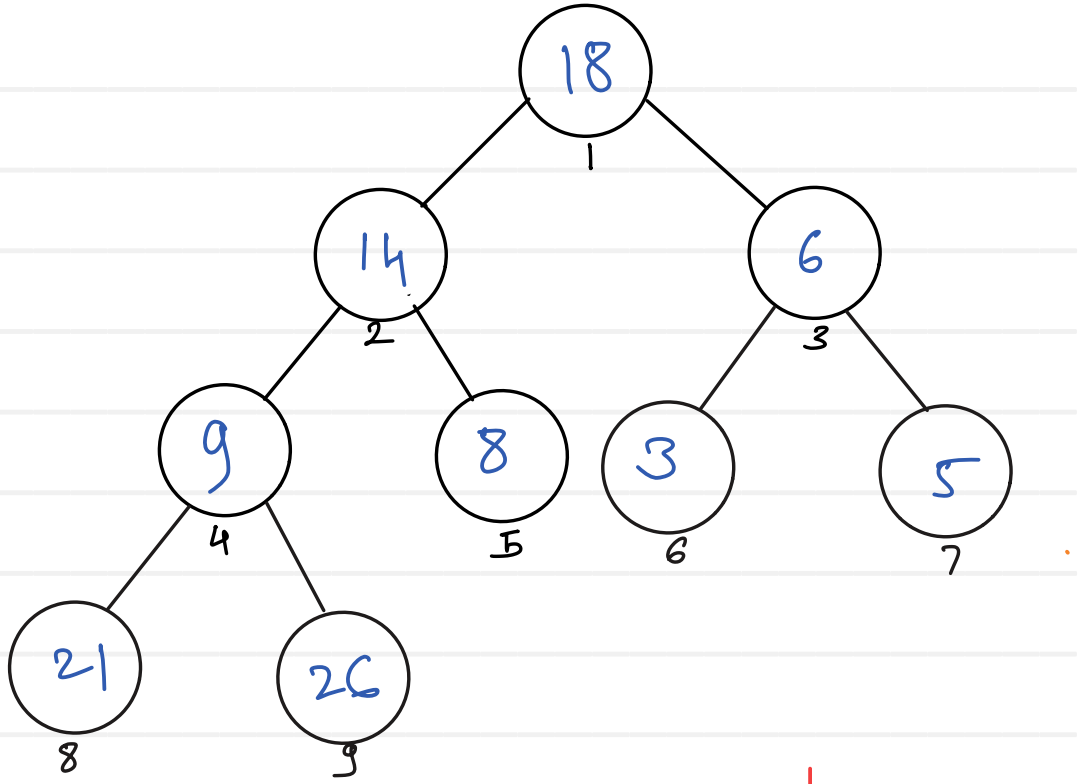
to create heap – $n \log n$

to delete heap – $n \log n$

Time $\propto 2n \log n$

$$T(n) = O(n \log n)$$

Best Avg worst



18	14	6	9	8	3	5	21	26
1	2	3	4	5	6	7	8	9